Hands-On Lab: Using `ConcurrentDictionary` in C#

Overview

In this lab, you'll learn how to use the `ConcurrentDictionary` from the `System.Collections.Concurrent` namespace. You will create a word frequency counter that processes multiple text files concurrently and stores the results in a thread-safe dictionary.

Prerequisites

- Basic understanding of C# and .NET.
- Familiarity with `Task`, `async`, and `await`.
- Visual Studio or a similar IDE.

Objectives

By the end of this lab, you will:

1. Understand the purpose and usage of `ConcurrentDictionary`.
2. Learn to safely update shared data structures in a multithreaded environment.
3. Build a word frequency counter that processes text files concurrently.

Step 1: Setup the Project

1. Open Visual Studio and create a new **Console App** project.
2. Name the project `ConcurrentDictionaryLab`.

Step 2: Add Sample Text Files

1. Create a folder named `SampleTexts` in the project directory.
2. Add a few `.txt` files with sample text. For example:
    - `file1.txt`:

```
Hello world. Welcome to the world of C# programming.
```

- `file2.txt`:

  ```
  C# makes concurrent programming fun and efficient.
  ```

- `file3.txt`:

  ```
  The world is full of opportunities to learn C#.
  ```

Step 3: Create the Word Counter

1. Add the following code to `Program.cs`:

```
using System;
using System.Collections.Concurrent;
using System.IO;
using System.Linq;
using System.Threading.Tasks;

class Program
{
    static async Task Main(string[] args)
    {
        var wordCounts = new ConcurrentDictionary<string, int>
(StringComparer.OrdinalIgnoreCase);

        // Get all text files in the SampleTexts directory
        var textFiles = Directory.GetFiles("SampleTexts", "*.txt");

        Console.WriteLine("Starting word count...");

        // Process files concurrently
        var tasks = textFiles.Select(file => Task.Run(() => ProcessFile(file,
wordCounts)));

        await Task.WhenAll(tasks);

        Console.WriteLine("Word count completed. Results:");

        // Display results
        foreach (var kvp in wordCounts.OrderByDescending(kv => kv.Value))
        {
            Console.WriteLine($"{kvp.Key}: {kvp.Value}");
        }
    }
```

```
    static void ProcessFile(string filePath, ConcurrentDictionary<string, int>
wordCounts)
    {
        Console.WriteLine($"Processing {Path.GetFileName(filePath)}...");

        var text = File.ReadAllText(filePath);
        var words = text.Split(new[] { ' ', '.', ',', ';', '!', '?' },
StringSplitOptions.RemoveEmptyEntries);

        foreach (var word in words)
        {
            wordCounts.AddOrUpdate(
                word,          // Key
                1,             // Value if the key does not exist
                (_, count) => count + 1 // Update logic if the key exists
            );
        }

        Console.WriteLine($"Finished processing {Path.GetFileName(filePath)}.");
    }
}
```

Step 4: Test the Application

1. Run the program.
2. Observe the output displaying the word frequencies from all the text files.

Step 5: Enhance the Word Counter

1. Add support for case-insensitive word counting (already implemented using `StringComparer.OrdinalIgnoreCase` in the `ConcurrentDictionary`).
2. Extend the program to handle large files:
    - Use `File.ReadLines` instead of `File.ReadAllText` for memory efficiency.

```
static void ProcessFile(string filePath, ConcurrentDictionary<string, int> wordCounts)
{
    Console.WriteLine($"Processing {Path.GetFileName(filePath)}...");

    foreach (var line in File.ReadLines(filePath))
    {
        var words = line.Split(new[] { ' ', '.', ',', ';', '!', '?' },
StringSplitOptions.RemoveEmptyEntries);

        foreach (var word in words)
```

```
        {
            wordCounts.AddOrUpdate(
                word,          // Key
                1,             // Value if the key does not exist
                (_, count) => count + 1 // Update logic if the key exists
            );
        }
    }

    Console.WriteLine($"Finished processing {Path.GetFileName(filePath)}.");
}
```

Step 6: Handle Exceptions

1. Add error handling to ensure robustness:

```
static void ProcessFile(string filePath, ConcurrentDictionary<string, int> wordCounts)
{
    try
    {
        Console.WriteLine($"Processing {Path.GetFileName(filePath)}...");

        foreach (var line in File.ReadLines(filePath))
        {
            var words = line.Split(new[] { ' ', '.', ',', ';', '!', '?' },
StringSplitOptions.RemoveEmptyEntries);

            foreach (var word in words)
            {
                wordCounts.AddOrUpdate(
                    word,          // Key
                    1,             // Value if the key does not exist
                    (_, count) => count + 1 // Update logic if the key exists
                );
            }
        }

        Console.WriteLine($"Finished processing {Path.GetFileName(filePath)}.");
    }
    catch (Exception ex)
    {
        Console.WriteLine($"Error processing {filePath}: {ex.Message}");
    }
}
```

Summary

In this lab, you:

1. Used a `ConcurrentDictionary` to count word frequencies across multiple files concurrently.
2. Leveraged `AddOrUpdate` to safely update the dictionary in a multithreaded environment.
3. Extended the program to handle large files and errors gracefully.